

C++ 프로그래밍 과제3

표준입출력과 파일 디스크립터 분석

■ 인 적 사 항

- 학부(과): 인공지능학부 인공지능전공
- 학 번: 20230841
- 이 름: 최준수

■ 이 론 설 명

- 파일 디스크립터란 무엇인가?

파일 디스크립터(FD, File Descriptor)란, Unix OS에서 네트워크 소켓이나 같은 파일이나 기타 입력/출력 리소스에 액세스하는 데 사용되는 추상표현이다. 즉, 시스템으로 부터 할당 받은 파일이나 소켓을 대표하는 정수이다. 마이크로소프트 윈도우와 C 표준 입출력 라이브러리 환경에서 “파일 핸들”(file handle)이라는 말이 선호되지만, 파일 디스크립터와는 기술적으로 다른 객체이다.

Unix OS에서 파일 디스크립터는 음이 아닌 정수로, 즉, C형 int를 말한다. 모든 프로세스가 갖춰야하는 표준 Unix OS 파일 디스크립터는 다음과 같이 세 개가 있다.

이름<stdio.h> file stream	내용	File Descriptor
stdin	표준 입력	0
stdout	표준 출력	1
stderr	표준 오류	2

파일 디스크립터는 커널이 관리하는 리소스에 대한 ‘핸들’ 역할을 하며, 프로세스별로 고유한 값을 가진다.

왜 파일 디스크립터는 음이 아닌 정수로 표현할까? 그 이유는 운영체제 커널의 효율성과 시스템 보안, 그리고 리소스 관리 때문이라고 할 수 있다.

우선 커널의 효율성 측면에서 보자. 커널 내부의 각 프로세스는 자신이 열어놓은 파일들을 관리하는 파일 서술자 테이블(File Descriptor Table)을 가지고 있다. 이 테이블은 실제로 Array 구조로 구현되어 있는데, 이때 파일 디스크립터가 음이 아닌 정수이기 때문에 File Descriptor Table에 인덱스(index)로 접근할 수 있다. 특정 파일의 정보에 접근할 때, 특별한 탐색 과정 없이 배열의 위치로 바로 찾아 갈 수 있으므로, 매우 빠르게 접근 가능하다.

또, 커널 자원의 보안 측면에서 보자. 음이 아닌 정수 파일 디스크립터는 실제 파일 데이터나 커널 내부의 복잡한 구조체를 프로세스로 부터 숨기는 추상적인 번호표 역할을 한다. 커널이 프로세스에게 접근 가능한 포인터를 직접 알려주는 대신, 파일 디스크립터의 추상적인 음이 아닌 정수만을 전달하고, 프로세스가 번호를 제시하면 커널이 대신 작업을 수행하게 된다.

추가적으로 리소스 관리 측면에서 보자. 정수는 리소스를 할당하고 회수하기에 매우 편리한 데이터 타입이다. Unix OS 계열 시스템은 내부적으로 새로운 파일을 열 때 사용하지 않는 가장 낮은 정수를 할당하는 규칙이 있다. 이는 리소스 번호를 촘촘하게 관리하여 메

모리 낭비를 줄인다.

POSIX 계열 운영체제(Unix, Linux, MacOS) 내부에서 파일 디스크립터는 운영체제와 프로세스가 대화하는 핵심 인터페이스 역할을 수행한다. 유닉스 계열의 가장 큰 특징은 하드웨어 장치, 네트워크 소켓, 파이프 등 모두 파일로 취급한 다는 것이다. 파일 디스크립터는 이 모든 이질적인 리소스들을 동일한 방식(read, write)으로 다룰 수 있게 해주는 추상화 도구이다. 즉, 개발자가 대상이 하드디스크의 파일인지, 키보드 입력인지, 아니면 멀리 떨어진 서버와의 통신인지 상관없이 동일한 API를 사용할 수 있게 한다.

그렇다면 Windows 운영체제에서는 이러한 리소스 식별 개념을 어떻게 다루고 있을까? Unix OS 계열이 파일 디스크립터라는 정수형 인덱스를 사용한다면, Windows OS는 앞서 언급한 파일 핸들이라는 보다 포괄적인 추상화 개념을 핵심으로 사용한다. Windows 환경에서 커널 오브젝트(파일, 프로세스, 스레드)에 접근하기 위해 사용하는 식별자는 Handle 타입이다. POSIX의 FD가 프로세스 내 배열의 인덱스인 것과 달리, Windowss의 핸들은 기술적으로 Void Pointer 형식을 취하게 된다. 앞서 Unix OS는 모든 것이 파일이다 라는 철학으로 운영된다고 언급했다. Windows는 그 철학 대신, 모든 리소스를 커널 오브젝트로 관리한다. 핸들은 이 오브젝트를 가리키는 간접적인 참조값이다.

Windows에서도 유닉스의 stdin, stdout, stderr와 대응되는 개념이 존재한다. 다만, 정수가 아닌 전용 API함수를 통해 리소스의 핸들을 획득한다.

이름	Windows 표준 핸들 상수	대응되는 POSIX FD
표준 입력	STD_INPUT_HANDLE	0 (stdin)
표준 출력	STD_OUTPUT_HANDLE	1 (stdout)
표준 오류	STD_ERROR_HANDLE	2 (stderr)

○ 표준입출력의 파일 디스크립터

앞서 Unix OS 계열에서 파일 디스크립터를 나타낸 표를 확인한 바가 있다. 그럼 각각이 무슨 역할을 하는지 확인하자.

이름<stdio.h> file stream	내용	File Descriptor
stdin	표준 입력	0
stdout	표준 출력	1
stderr	표준 오류	2

표준 입력(stdin)은 프로세스가 데이터를 읽어들이는 기본 통로다. 기본적으로 키보드와 연결되어 있으며, 리다이렉션을 통해 파일이나 다른 프로그램의 출력을 입력받을 수 있다.

표준 출력(stdout)은 프로세스가 정상적인 실행 결과를 내보내는 기본 통로다. 기본적으로 터미널 화면(모니터)과 연결되어 있습니다. 표준 입력이 리다이렉션을 통해 파일이나 다른 프로그램의 출력을 입력받을 수 있다고 언급했다. 표준 출력은 반대로 리다이렉션을 통해 파일이나 다른 프로그램의 입력으로 출력할 수 있다.

표준 오류(stderr)은 로그램 실행 중 발생하는 에러 메시지나 진단 메시지를 출력하는 통로다. stdout과 별개로 분리되어 있어, 정상 결과는 파일에 저장하고 에러만 화면에 띄우는 등의 제어가 가능하다.

표준입출력의 파일 디스크립터는 프로그램이 시작될 때 이미 열려있다. 그럼 왜 프로그램 시작 시 왜 이미 열려 있을까? 그 이유는 바로 프로세스 상속(Inheritance)와 표준 환경의 약속 때문이다. 만약 이 서술자들이 미리 열려 있지 않다면, 모든 개발자는 printf를 하기 전에 터미널에 장치를 직접 open()하는 코드를 매번 작성해야한다. 이를 방지하기 위해 OS 환경은 프로세스가 태어날 때부터 기본적인 통로를 제공한다. Unix OS 계열에서 새로운 프로세스는 부모 프로세스를 복제하여 생성됩니다. 이때 부모가 가진 파일 서술자 테이블도 자식에게 그대로 복사됩니다. 즉, 부모인 셸(Shell)이 이미 열어놓은 통로를 자식이 그대로 물려받는 것입니다. 그럼 누가 표준입출력을 생성할까? 이들을 실제로 준비하고 연결하는 것은 Shell이고, 그 매커니즘을 제공하는 것은 OS이다.

○ C/C++ IO와 파일 디스크립터

C++의 iostream, C의 stdio, 그리고 운영체제의 파일 디스크립터는 독립된 시스템이 아니라 계층적 추상화 관계를 갖는다. 가장 하단에는 운영체제가 리소스를 식별하기 위해 사용하는 파일 디스크립터가 존재한다. C 표준 라이브러리는 이 FD를 한 번 더 감싸서 파일 스트림(FILE*) 구조체로 관리하고, C++은 이를 다시 객체 지향적으로 래핑하여 iostream 객체를 제공하는 구조이다.

C++	C	OS
std::cin	stdin	fd 0
std::cout	stdout	fd 1
std::cerr	stderr	fd 2

즉, std::cout은 내부적으로 C의 stdout을 참조하며, stdout은 최종적으로 운영체제의 fd 1을 가리키게 된다. 이러한 계층 구조 덕분에 상위 라이브러리로 갈수록 개발자는 저수준의 복잡한 시스템 호출을 직접 다루지 않고도 편리하게 입출력을 수행할 수 있다.

printf와 cout은 데이터를 출력한다는 목적은 같지만, 내부적으로 이를 처리하는 방식과 설계 철학에서 큰 차이가 있다. printf는 가변 인자를 사용하는 함수 형태로 구현되어 있다. 또, 실행 시점에 포맷 문자열(예: %d, %f)을 해석하여 데이터 타입을 판단한다. 이 과정에서 타입이 맞지 않으면 런타임 오류가 발생할 수 있다. 내부적으로 FILE 구조체에 포함된 버퍼를 사용하며, 특정 조건이 만족될 때 write() 시스템 콜을 호출한다. Cout은 연산자 오버로딩(<<)을 사용하여 컴파일 시점에 데이터 타입이 이미 결정된다. 따라서 printf보다 타입 안전성(Type Safety)이 높고 속도 면에서 유리할 수 있다.

결론적으로, 두 방식 모두 결국 운영체제의 파일 디스크립터를 통해 데이터를 전달하지만, C++의 cout은 객체 지향적인 래핑을 통해 더 안전하고 유연한 인터페이스를 제공하고 할 수 있다.

■ 실험 1

○ stdout/stderr 리다이렉션 실험

```
#include<iostream>

int main(void){
    std::cout << "This is stdout message" << std::endl;
    std::cerr << "This is stderr message" << std::endl;

    return 0;
}
```

다음 프로그램은 C++ 언어로 cout과 cerr의 출력차이를 확인 할 수 있는 코드이다. 이 프로그램을 기본 실행하게 되면 다음과 같은 결과 값을 얻을 수 있다.

```
PS C:\Users\Admin\code\cpp-programming\Homework\2.Standard IO and File Descriptors> g++ .\stdoutstderrRedirection.cpp -o main
PS C:\Users\Admin\code\cpp-programming\Homework\2.Standard IO and File Descriptors> ls

Directory: C:\Users\Admin\code\cpp-programming\Homework\2.Standard IO and File Descriptors

Mode                LastWriteTime         Length Name
----                -
-a----          2026-04-15 오후 11:30         131872 20230841_최준수_표준입출력보고서.hwp
-a----          2026-04-15 오후 11:34         138199 main.exe
-a----          2026-04-15 오후 11:34           158 stdoutstderrRedirection.cpp

PS C:\Users\Admin\code\cpp-programming\Homework\2.Standard IO and File Descriptors> ./main
This is stdout message
This is stderr message
PS C:\Users\Admin\code\cpp-programming\Homework\2.Standard IO and File Descriptors> |
```

분명 std::cerr와 std::cout으로 서로 다른 표준입출력을 사용했지만, 사용자의 화면(터미널) 상에는 std::cerr와 std::cout의 출력에는 큰 차이를 확인할 수 없었다. 그럼 해당 프로그램의 표준입출력을 리다이렉션하여 std::cout의 결과값과 std::cerr의 결과값을 txt파일로 저장해서 확인해보자.

```
stdoutstderrRedirection.cpp U  out.txt U X
Homework > 2.Standard IO and File Descriptors > out.txt
1 This is stdout message
2

문제 출력 디버그 콘솔 터미널 포트

PS C:\Users\Admin\code\cpp-programming\Homework\2.Standard IO and File Descriptors> ./main > out.txt
This is stdout message
PS C:\Users\Admin\code\cpp-programming\Homework\2.Standard IO and File Descriptors> ls

Directory: C:\Users\Admin\code\cpp-programming\Homework\2.Standard IO and File Descriptors

Mode                LastWriteTime         Length Name
----                -
-a----          2026-04-15 오후 11:42         188224 20230841_최준수_표준입출력보고서.hwp
-a----          2026-04-15 오후 11:34         138199 main.exe
-a----          2026-04-15 오후 11:44           50 out.txt
-a----          2026-04-15 오후 11:35         29630 stdoutStderr.png
-a----          2026-04-15 오후 11:36           171 stdoutstderrRedirection.cpp

PS C:\Users\Admin\code\cpp-programming\Homework\2.Standard IO and File Descriptors> |
```

./main > out.txt를 통해 std::cout의 출력 결과를 out.txt로 출력해줬다. 그 결과 동일 디렉토리에 out.txt라는 새로운 파일이 생성되었고, out.txt라는 파일 속에 코드에서 작성한 “This is stdout message”라는 문자열이 생성되었다.

std::cout은 txt 파일로 출력되어 나갔다. 그럼 std::cerr는 어떻게 되었을까? 사진을 보면 알 수 있듯, 기존에 화면(터미널)에 출력되는 방식과 동일하게 출력된 것을 확인할 수 있다. 사용자가 std::cerr의 출력 방식을 정해주지 않았기 때문에 기존의 표준 입출력에 따라서 출력된 것으로 예상할 수 있다.

```
HomeWork > 2.Standard IO and File Descriptors > err.txt
1  ./main : This is stderr message
2  At line:1 char:1
3  + ./main 2> err.txt
4  + ~~~~~
5  + CategoryInfo          : NotSpecified: (This is stderr message:String) [], RemoteException
6  + FullyQualifiedErrorId : NativeCommandError
7

문제 출력 디버그 콘솔 터미널 포트

PS C:\Users\Admin\code\cpp-programming\HomeWork\2.Standard IO and File Descriptors> ./main 2> err.txt
This is stdout message
PS C:\Users\Admin\code\cpp-programming\HomeWork\2.Standard IO and File Descriptors> ls

Directory: C:\Users\Admin\code\cpp-programming\HomeWork\2.Standard IO and File Descriptors

Mode                LastWriteTime         Length Name
----                -
-a----            2026-04-15 오후 11:57        229376 20230841_최준수_표준입출력보고서.hwp
-a----            2026-04-15 오후 11:57          488 err.txt
-a----            2026-04-15 오후 11:34       138199 main.exe
-a----            2026-04-15 오후 11:44           50 out.txt
-a----            2026-04-15 오후 11:35       29630 stdoutStderr.png
-a----            2026-04-15 오후 11:36         171 stdoutstderrRedirection.cpp
```

그럼 이어서 stderr를 리다이렉션 해보자. 앞서 std::cout을 텍스트파일로 리다이렉션한 것과 동일하게, stderr도 잘 리다이렉션된 것을 확인할 수 있다. 눈치가 빠른 사람이라면 std::cout을 리다이렉션할 때와 std::cerr를 리다이렉션할 때 사용한 명령어가 다른것을 알 수 있을 것이다. 그 이유는 std::cout과 다르게, std::cerr는 표준 에러 통로를 지목해야하기 때문이다. 즉, ./main 2> err.txt에서 숫자 2의 의미는 표준 에러 통로를 지목한다는 것이다. 결과적으로, 프로그램 실행 중 발생한 에러 메시지만 err.txt 파일에 따로 저장하고, 정상적인 출력(stdout)은 그대로 터미널에 보여준다.

```
1  This is stdout message
2  ./main : This is stderr message
3  At line:1 char:1
4  + ./main > all.txt 2>&1
5  + ~~~~~
6  + CategoryInfo          : NotSpecified: (This is stderr message:String) [], RemoteException
7  + FullyQualifiedErrorId : NativeCommandError

문제 출력 디버그 콘솔 터미널 포트

PS C:\Users\Admin\code\cpp-programming\HomeWork\2.Standard IO and File Descriptors> ./main > all.txt 2>&1
PS C:\Users\Admin\code\cpp-programming\HomeWork\2.Standard IO and File Descriptors> ls

Directory: C:\Users\Admin\code\cpp-programming\HomeWork\2.Standard IO and File Descriptors

Mode                LastWriteTime         Length Name
----                -
-a----            2026-04-16 오전 12:01        262144 20230841_최준수_표준입출력보고서.hwp
-a----            2026-04-16 오전 12:02          552 all.txt
-a----            2026-04-15 오후 11:57          488 err.txt
-a----            2026-04-15 오후 11:34       138199 main.exe
-a----            2026-04-15 오후 11:44           50 out.txt
-a----            2026-04-15 오후 11:35       29630 stdoutStderr.png
-a----            2026-04-15 오후 11:36         171 stdoutstderrRedirection.cpp
```

마지막으로 stdout과 stderr를 하나로 합쳐보자. 두 출력을 합칠 때 사용한 명령어는 다음과 같다. ./main > all.txt 2>&1 이 명령어에서 특이한 점을 뽑으라고 한다면 아마 다들 2>&1를 뽑을 것이다. 2>&1은 파일 디스크립터의 복제(Duplicate)를 의미하는 아주 중요한 구문이다. 특히 &는 뒤에 오는 숫자가 파일 이름이 아니라 파일 디스크립터 번호임을 셸에게 알려주는 매우 중요한 역할을 한다. 만약, 파일 디스크립터 관점에서 &가 없다면 1이라는 이름의 파일을 새로 만들게 될것이다.

■ 실험 2

○ stdin 리다이렉션 실험

```
2.Standard IO and File Descriptors > read_input.cpp > main(void)
1  #include<iostream>
2  #include<string>
3
4
5  int main(void){
6      std::string line;
7      std::cout<<"Reading from STDIN. . ."<<std::endl;
8      while(std::getline(std::cin, line)){
9          std::cout << "Input : " << line << std::endl;
10     }
11 }
```

stdin 리다이렉션 실행을 위한 코드를 작성했다. 우선 처음으로는 이 코드를 평범하게 실행하여 사용자의 터미널 상에서 입력을 받도록 하겠다.

```
문제 출력 디버그 콘솔 터미널 포트
PS C:\Users\Admin\code\cpp-programming\Homework\2.Standard IO and File Descriptors> .\read_input.exe
Reading from STDIN. . .
apple
Input : apple
banana
Input : banana
orange
Input : orange
PS C:\Users\Admin\code\cpp-programming\Homework\2.Standard IO and File Descriptors> []
```

input.txt에 미리 프로그램의 input을 작성한 뒤, read_input < input.txt를 통해 프로그램에 입력을 넣어보자.

```
2.Standard IO and File Descriptors > input.txt
1  apple
2  banana
3  orange

명령 프롬프트
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Admin>cd C:\Users\Admin\code\cpp-programming\Homework\2.Standard IO and File Descriptors
C:\Users\Admin\code\cpp-programming\Homework\2.Standard IO and File Descriptors>./read_input.exe < input.txt
'.' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\Admin\code\cpp-programming\Homework\2.Standard IO and File Descriptors>read_input.exe < input.txt
Reading from STDIN. . .
Input : apple
Input : banana
Input : orange
C:\Users\Admin\code\cpp-programming\Homework\2.Standard IO and File Descriptors>
```

첨부한 사진을 확인하면 알 수 듯, 코드를 수정하지 않았는데도, 입력파일을 잘 입력받는 이유는 무엇일까? 그 비밀은 <input.txt에 있다. 일반적으로 std::cin은 키보드 입력을 기다린다. read_input.exe < input.txt라고 입력하면, 운영체제가 "이 프로그램의 표준 입력(stdin)을 키보드가 아니라 input.txt 파일로 연결해!"라고 명령하는 것이다. 즉, OS의 똑똑한 처리 덕분에 코드를 수정하지 않고도 외부 파일을 입력할 수 있는 것이다.


```
Admin@DESKTOP-I6MB7EI MINGW64 ~/code/cpp-programming/HomeWork/2.Standard IO and File Descriptors (main)
$ cat input.txt | ./read_input.exe
Reading from STDIN. . .
Input : apple
Input : banana
Input : orange
```

다음 코드는 pipe로 input.txt를 read_input으로 넘겨준 것이다. pipe는 앞에 실행한 명령어의 출력을 다음에 오는 명령어의 입력으로 넘겨주는 명령어이다. cat input.txt는 파일을 읽어서 내용을 자신의 표준 출력(stdout)으로 내보낸다. 이때 pipe는 앞 프로그램의 stdout을 뒤 프로그램의 stdin으로 직접 연결하게 된다. 결과적으로 read_input.exe는 그 통로를 타고 넘어오는 데이터를 입력으로 받게 된다.

파일 디스크립터 관점에서는 셸이 |를 입력받게 된다면, pipe() 시스템 콜을 호출하여 데이터를 임시로 저장할 수 있는 커널 버퍼를 만들게 된다. 그 뒤, cat 프로세스의 FD 1번(stdout)을 파이프의 쓰기 종단에 연결한다. 이어서 read_input.exe 프로세스의 FD 0번(stdin)을 파이프의 읽기 종단에 연결한다. 즉, cat이 FD 1번에 데이터를 쓰면, 커널 버퍼를 거쳐 read_input.exe의 FD 0번으로 데이터가 전달된다.

■ 분석 및 결론

본 실험을 통해 C++ 언어의 표준 입출력 스트림이 OS의 파일 디스크립터와 상호작용하는 방식을 확인하였다. 우선 프로그램은 물리적인 파일을 직접 읽는 것이 아니라 OS가 프로세스 생성 시 부여한 0번 파일 디스크립터인 stdin 스트림을 읽는다. 이러한 추상화된 통로 덕분에 소스 코드 수정 없이 입력 소스를 키보드나 파일로 자유롭게 변경할 수 있는 유연성을 갖게 된다.

또한 표준 입출력인 stdin, stdout, stderr의 명확한 역할 분담을 체득하였다. stdin (Standard Input, FD 0)은 데이터 유입의 기본 입구로 Shell을 통해 연결되며, stdout (Standard Output, FD 1)은 정상 결과를 내보내는 주 출구이다. stderr (Standard Error, FD 2)은 오류 및 진단 메시지를 위한 별도의 전용 통로이다. 실험 1을 통해 stdout과 stderr를 분리하여 txt파일로 저장해 봄으로써 정상 데이터와 에러 로그를 독립적으로 관리하는 시스템 프로그래밍의 원칙을 이해하였다. 특히 2>&1과 같은 구문은 OS 레벨에서 파일 디스크립터가 복제되어 서로 다른 표준 통로가 합쳐지는 과정을 보여주었다.

더불어 pipe를 통한 프로세스 간 통신의 원리를 파악하였다. cat의 표준 출력을 pipe로 연결하는 과정은 커널 버퍼와 파일 디스크립터 시스템이 유기적으로 작동한 결과이며, 이는 작은 도구들을 연결해 시스템을 구축하는 Unix 철학을 실천적으로 확인한 계기가 되었다. 결과적으로 C++ 언어 프로그래밍에서 입출력은 OS 리소스 관리 시스템과 밀접하게 맞물려 작동하며, 파일 디스크립터에 대한 이해는 향후 데이터 흐름을 효율적으로 제어하기 위한 필수적인 토대가 될 것이다.